

Instituto Federal de Goiás — IFG
Disciplina: Programação Orientada a Objetos
Avaliação Prática

O sistema acadêmico já possui a entidade **Curso** implementada no back-end (Spring Boot) e no front-end (Angular). O repositório de referência com o código de Curso está disponível em:

<https://github.com/DoryGR/WebService-RestFull-2026/tree/v1.0-prova>

<https://github.com/DoryGR/Academico-Fontend-2026>

Sua tarefa é implementar a entidade **Aluno** seguindo exatamente o mesmo padrão.

Ambiente: back-end na porta **8080** com context path **/sistema-academico**
 front-end na porta **4200**.

A entidade Aluno possui os seguintes atributos:

```
@Id @GeneratedValue
private Integer idaluno;
private String nome;
private String sexo;
private Date dt_nasc;
@JsonIgnore @ManyToMany(mappedBy = "alunos")
private List<Curso> cursos;
```

O `AlunoRepository` já existe e estende `JpaRepository<Aluno, Integer>`.

1. Back-end: AlunoResource

Crie a classe `AlunoResource` no pacote `resources` com os cinco endpoints abaixo, seguindo o padrão de `CursoResource`:

- GET `/alunos` — retorna todos os alunos (200 OK)
- GET `/alunos/{id}` — retorna um aluno pelo id (200 OK)
- POST `/alunos` — cadastra um novo aluno (201 Created, header Location preenchido)
- PUT `/alunos/{id}` — atualiza um aluno existente (200 OK)
- DELETE `/alunos/{id}` — remove um aluno (204 No Content)

A classe deve ter `@RestController`, `@RequestMapping("/alunos")` e `@CrossOrigin("http://localhost:4200")`.

O `AlunoService` deve ser injetado via construtor.

2. Back-end: AlunoService

Crie a classe AlunoService no pacote services, anotada com @Service, com os métodos findAll, findById, insert, update e delete. O método findById deve lançar ResourceNotFoundException quando o id não existir.

3. Front-end: Model

Crie o arquivo src/app/models/aluno.ts com a interface TypeScript correspondente à entidade. O campo idaluno deve ser opcional.

4. Front-end: AlunoService

Crie o serviço src/app/services/aluno.service.ts com os métodos getAll, getById, create, update e delete, consumindo a URL base *http://localhost:8080/sistema-academico/alunos*.

5. Front-end: AlunoListComponent

Crie o componente src/app/components/aluno-list com uma tabela exibindo nome, sexo e dt_nasc. O componente deve carregar os alunos no ngOnInit, ter botão Editar (navega para /alunos/editar/:id), botão Excluir (remove e atualiza a tabela) e botão Novo Aluno (navega para /alunos/novo).

6. Front-end: AlunoFormComponent

Crie o componente src/app/components/aluno-form com formulário reativo contendo os campos nome, sexo (mat-select com opções M e F) e dt_nasc (input type date). Quando a rota contiver um id, o formulário deve ser pré-preenchido com patchValue. Ao salvar, redirecionar para /alunos.

7. Front-end: Rotas

Adicione em app.routes.ts as rotas /alunos, /alunos/novo e /alunos/editar/:id, sem remover as rotas de Curso existentes.

Pontuação:

	Descrição	Pontuação
1	AlunoResource — endpoints e anotações	0,90
2	AlunoService — métodos e exceção	0,45
3	Model aluno.ts — interface TypeScript	0,15
4	AlunoService Angular — métodos e URL base	0,45
5	AlunoListComponent — tabela, exclusão e navegação	0,60
6	AlunoFormComponent — formulário e edição	0,30
7	Rotas — sem quebrar as de Curso	0,15
	Total	3,0 pontos

Entregar o link do repositório Git com o código do back-end e do front-end. via e-mail:
dory.rodrigues@ifg.edu.br